

CPU 사용이 많은 곳 정리

주문·결제·정산 MSA 플랫폼에서 **CPU 를 많이 쓰는 코드 경로**를 실제 코드 근거와 함께 정리한 문서. "왜 CPU 를 쓰는지 → 어디서 도는지(파일·라인) → 빈도 → 완화 여지" 순으로 기술한다.

표기: 🌀 상시(주기 실행) · 📧 요청마다 · 📦 배치/대량 · 🧊 opt-in(기본 off) 본 문서는 **정적 코드 분석** 기반이며, 절대 수치는 프로파일러(JFR/async-profiler)로 별도 측정 필요.

한눈에 보기

#	핫스팟	위치	빈도	CPU 원인
①	BCrypt(cost=12) 해싱	<code>shared-common</code> SecurityConfig	📧 로그인/가입	$2^{12}=4096$ 라운드 키 스트레칭 (의도적 고비용)
②	JWT HS256 서명/검증	<code>shared-common</code> JwtUtil/Filter	📧 인증 요청마다	HMAC-SHA256 + 토큰 파싱
③	PDF 생성(iText)	settlement PDF 어댑터 2종	📧 정산서/리포트 요청	CJK 폰트 임베딩 + 레이아웃 계산
④	PII 마스킹 정규식	<code>shared-common</code> PIIMaskingConverter	📧 로그 라인마다	email/phone/card 정규식 3종 매칭
⑤	AOP 메서드 트레이싱	<code>shared-common</code> MethodTraceAspect	📧 web/service/kafka 호출마다	around advice + Timer 태그 등록
⑥	PG 대사 매칭 + CSV 파싱	<code>pgreconciliation</code>	📦 대사 실행	행별 split/BigDecimal/parse + 다중 HashMap
⑦	정산 일괄 생성	<code>CreateDailySettlementsService</code>	📦 🌀 매일 02시	결제 건별 수수료 BigDecimal 계산
⑧	ES 색인(유사-bulk)	<code>SettlementSearchAdapter</code>	📦 🧊	문서 매핑 + 단건 save N회
⑨	2-tier 캐시 직렬화	<code>TwoTierCacheConfig</code>	📧 🧊	Jackson <code>DefaultTyping.EVERYTHING</code>
⑩	Outbox 풀러	<code>OutboxPublisherScheduler</code>	🌀 2초 주기	카운트 쿼리 + 배치 직렬화/발행
⑪	Cashflow/Ledger 집계	<code>report,ledger</code> 도메인	📧 📦	BigDecimal 대량 연산·대사 3종

① BCrypt(cost=12) 비밀번호 해싱 — 단일 최대 CPU 핫스팟 📧

원인: BCrypt 는 의도적으로 느린 적응형 해시다. `cost=12` 는 $2^{12} = 4096$ 라운드의 키 스트레칭을 수행한다. 비밀번호 1건 해싱/검증에 수십 ms 의 순수 CPU 가 들며, 이는 설계상 의도된 비용(브루트포스 방어)이다.

위치: `shared-common/.../common/config/jwt/SecurityConfig.java:37`

```
return new BCryptPasswordEncoder(12);
```

빈도: 회원가입(POST /users), 로그인(POST /auth/login), 비밀번호 재설정마다 1회. 로그인 트래픽이 몰리면 인증 워커 CPU 가 가장 먼저 포화된다.

완화 여지:

- cost 는 낮추지 말 것(보안 저하). 대신 로그인 엔드포인트에 동시 실행 제한/큐잉을 두어 CPU 스파이크를 평탄화.
- 가상 스레드가 켜져 있어(spring.threads.virtual.enabled=true) 스레드는 무제한이지만, BCrypt 는 CPU-bound 라 가상 스레드로 처리량이 늘지 않는다 → CPU 코어 수가 진짜 상한.

② JWT HS256 서명/검증

원인: 토큰 발급 시 HMAC-SHA256 서명, 모든 인증 요청에서 서명 검증 + 클레임 파싱. BCrypt 보다는 가볍지만 **모든 보호된 요청 경로**를 통과하므로 누적 비용이 크다.

위치:

- shared-common/.../jwt/JwtUtil.java:43,51 — signWith / parseSignedClaims
- shared-common/.../jwt/JwtAuthenticationFilter.java:44 — 요청당 parseToken 1회

이미 적용된 최적화: 필터에서 기존 validateToken + getEmail + parseToken 3회 파싱을 1회로 통합 (JwtAuthenticationFilter.java:43 주석). 헬스/스웨거/로그인 경로는 shouldNotFilter 로 검증 스킵.

완화 여지: 키 객체(SecretKey)는 생성자에서 1회 생성·재사용 중(양호). 추가 최적화는 검증 결과 캐싱 정도이나 stateless 원칙과 상충 → 현 구조 유지 권장.

③ PDF 생성 (iText 8) — 요청당 무거운 연산

원인: PDF 렌더링은 문서 트리 빌드 + 레이아웃 계산 + **CJK 폰트 임베딩**으로 요청당 CPU-메모리를 크게 쓴다. 특히 한국어 폰트(HYGoThic-Medium, HYSMyeongJo-Medium)를 렌더 메서드 안에서 매번 PdfFontFactory.createFont(...) 로 로드한다.

위치:

- settlement-service/.../settlement/adaptor/out/pdf/SettlementPdfAdaptor.java:65-68 — render() 호출마다 폰트 2종 새로 생성
- settlement-service/.../report/adaptor/out/pdf/CashflowPdfAdaptor.java — 캐시플로우 리포트 PDF

```
PdfFont fontRegular = PdfFontFactory.createFont(
    "HYGoThic-Medium", "UniKS-UCS2-H", EmbeddingStrategy.PREFER_EMBEDDED); // 매
render 마다
```

완화 여지 (개선 후보):

- 폰트 객체는 무겁지만 사실상 불변 → **정적/싱글턴으로 1회 로드 후 재사용** 가능 (단, iText PdfFont 가 PdfDocument 에 바인딩되는지 버전별 확인 필요).
- 대량 일괄 발급은 요청 스레드 밖(배치/비동기)으로 분리하고 동시 생성 수를 제한.

④ PII 마스킹 정규식 — 로그 라인마다

원인: logback converter 가 모든 로그 메시지에 대해 email/phone/card 정규식 3종을 순차 매칭한다. 로그량이 많은 경로(요청 트레이싱 ⑤와 결합 시)에서 누적 CPU 가 된다.

위치: shared-common/.../common/config/observability/PIIMaskingConverter.java:48-95

```
result = maskEmails(result);    // EMAIL_PATTERN
result = maskPhones(result);    // PHONE_PATTERN
result = maskCards(result);    // CARD_PATTERN (+ replaceAll, repeat)
```

양호한 점: Pattern 은 static final 로 사전 컴파일(매칭 시 재컴파일 없음).

완화 여지: 로그 레벨을 운영에서 INFO 이상으로 유지(DEBUG 트레이스 억제)하면 마스킹 호출 횟수가 급감. 카드 패턴은 `\b...\d{4}` 4그룹이라 가장 무거움 → 카드번호가 로그에 거의 없다면 해당 마스킹만 조건부 비활성 고려.

⑤ AOP 메서드 트레이싱

원인: web 어댑터·application 서비스·Kafka 컨슈머 모든 메서드 호출을 around advice 로 감싼다. 호출마다 시그니처 파싱, `System.nanoTime()` 2회, Micrometer Timer 태그 등록/조회, 그리고 log-args 켜진 경우 인자 전체 `String.valueOf` + stream join.

위치: shared-common/.../common/observability/aop/MethodTraceAspect.java:52-89 (포인트컷 `LemuelPointcuts.traceable()`)

양호한 점: DEBUG 로그는 `log.isDebugEnabled()` 가드, log-args 는 토글, 인자 길이 상한 (`maxArgLength`)으로 과도한 문자열화 방지.

완화 여지: 고빈도 경로(예: 조회 컨트롤러)는 포인트컷에서 제외하거나, 인자 로깅은 항상 off 유지. Timer 태그의 `method/class` 카디널리티가 높으면 메트릭 저장 비용도 증가하니 주의.

⑥ PG 대사 매칭 + CSV 파싱 — 대용량 입력 시

원인 (파싱): PG 정산파일을 행마다 `line.split(",")` → 컬럼 4개를 `new BigDecimal(...), LocalDate.parse(...)` 로 변환. 파일이 수십만 행이면 객체 생성·파싱이 CPU 를 점유.

원인 (매칭): 양방향 대사를 위해 PG/내부 행을 여러 **HashMap** 으로 인덱싱하고, 매칭 건마다 `BigDecimal.subtract().abs().compareTo()` 를 수행. O(n) 이지만 BigDecimal 연산 상수가 크다.

위치:

- `pgreconciliation/adaptor/out/file/CsvPgFileParserAdapter.java:51-66` — 행별 파싱 루프
- `pgreconciliation/domain/PgReconciliationMatcher.java:32-110` — 4개 컬렉션 빌드 + 행별 비교

완화 여지: 알고리즘은 적절(HashMap O(1) 조회). 초대용량이면 스트리밍 파싱 + 청크 단위 대사, 혹은 DB 집계로 1차 필터 후 차이분만 인메모리 대조.

⑦ 정산 일괄 생성 — 매일 새벽 배치

원인: 전일 승인 결제 전 건을 stream 으로 돌며 `Settlement.createFromPayment(...)` (수수료.net 금액 **BigDecimal** 계산) 후 건별 저장. 건수에 비례해 CPU·DB 라운드트립 증가.

위치:

- `settlement-service/.../settlement/application/service/CreateDailySettlementsService.java:43-64`
- 트리거: `SettlementScheduler.java:31` (@Scheduled(cron = "0 0 2 * * *"), ShedLock 으로 1노드만)

양호한 점: 단일 노드에서만 실행(ShedLock) → 중복 CPU 없음. 가상 스레드 영향 없음(CPU-bound).

완화 여지: 건별 `saveSettlementPort::save` 대신 **JDBC 배치 저장**(이미 `tps.md` ③ 에서 `reWriteBatchedInserts` + `jdbc.batch_size` 적용)으로 DB 비용 절감. ES 색인은 try/catch 로 분리되어 배치 본류를 막지 않음.

⑧ Elasticsearch 색인 (이름만 bulk)

원인: `bulkIndexSettlements` 가 도메인→문서 매핑 후 `documents.forEach(save)` 로 **단건 색인을 N회** 호출한다(진짜 `_bulk` API 아님). 건수만큼 직렬화·HTTP 왕복 발생.

위치: `settlement-service/.../settlement/adapters/out/search/SettlementSearchAdapter.java:42-53`

```
documents.forEach(elasticsearchOperations::save); // ← 진짜 bulk 가 아닌 단건 N회
```

빈도: `app.search.enabled=true` 일 때만(기본 off). 일일 정산 생성 직후 호출.

완화 여지 (개선 후보): Spring Data ES 의 `save(Iterable)` 또는 `BulkOperations` 로 묶어 **실제 _bulk** 한 번에 전송하면 직렬화/네트워크 비용이 크게 준다.

⑨ 2-tier 캐시 Jackson 직렬화

원인: L2(Redis) 직렬화에 `DefaultTyping.EVERYTHING` 을 켜서 **final 스칼라(Long/BigDecimal) 까지 타입 정보 (@class)를 부여**한다. round-trip 충실성(예: `List<Long>` 이 Integer 로 변질 방지)을 얻는 대신, 직렬화 노드 수와 출력 크기가 늘어 CPU·대역폭 비용이 증가한다.

위치: `shared-common/.../common/config/cache/TwoTierCacheConfig.java:86`

```
om.activateDefaultTyping(ptv, ObjectMapper.DefaultTyping.EVERYTHING,
JsonTypeInfo.As.PROPERTY);
```

빈도: `app.cache.two-tier.enabled=true` 이고 Redis 가 있을 때만(기본 off). 캐시 쓰기/L2 적중마다.

완화 여지: 타입 충실성과 성능의 트레이드오프. 필요한 캐시값에만 `EVERYTHING` 대신 `NON_FINAL` 적용 검토 가능하나, `List<Long>` 변질 방지 요건과 충돌하므로 신중히.

⑩ Outbox 발행 폴러 — 상시 주기

원인: 2초마다 (1) `countPending/countFailed` 카운트 쿼리 2회 (2) `FOR UPDATE SKIP LOCKED` claim (3) 배치 이벤트 **직렬화 + Kafka 발행**. 발행 자체보다 주기적 카운트/직렬화가 상시 CPU 를 먹는다.

위치: `shared-common/.../common/outbox/application/service/OutboxPublisherScheduler.java:72-90`
`(@Scheduled(fixedDelayString = "${app.outbox.polling-delay-ms:2000}"))`

양호한 점: 이미 멀티워커 배치 발행으로 최적화(`tps.md` ④⑤). `ShedLock` 제거 → 인스턴스 수만큼 병렬 확장. 빈 배치면 즉시 return.

완화 여지: 한가한 시간대엔 폴링 주기(`app.outbox.polling-delay-ms`)를 늘려 idle CPU 절감. `ledger` 도 별도 폴러 존재 (`LedgerOutboxPoller.java:36`, 5초 주기) — 동일하게 적용 가능.

⑪ Cashflow / Ledger 집계 — BigDecimal 대량 연산

원인: 캐시플로우 리포트는 집계 + **대사(reconciliation) 3종 불변식 검증**을 수행하며, 각 체크가

`BigDecimal.subtract/add/compareTo` 를 반복한다. 복식부기 원장(ledger)도 차변/대변 합산에 `BigDecimal` 을 광범위 하게 쓴다(`BigDecimal` 사용처 88개 파일).

위치:

- `settlement-service/.../report/application/service/GenerateCashflowReportService.java:108-160` — 대사 3종(`payments-refunds=net+commission` 등)
- `settlement-service/.../ledger/...` — 원장 기입/역분개

양호한 점: 집계 자체는 대부분 **DB 쿼리(SUM)** 로 위임(`LoadPeriodReconciliationPort`), 인메모리 `BigDecimal` 은 최종 비교 수준 → 무겁지 않음. `Timer` 는 생성자에서 1회 생성-재사용.

완화 여지: 현 구조 양호. 판매자 단위 리포트는 대사를 건너뛰어(`isSellerScoped`) 불필요한 연산 제거됨.

종합 — 우선순위

우선순 위	항목	이유
● 높음	① BCrypt	단일 연산 비용 최대, 로그인 폭주 시 코어 포화. 큐잉/동시성 제한으로 대응(cost 유지)
● 중간	③ PDF 폰트 매 호출 로 드	폰트 싱글턴화로 즉시 개선 가능
● 중간	⑧ ES 유사-bulk	진짜 <code>_bulk</code> 전환으로 직렬화/양복 절감
● 낮음	④⑤ 마스킹·AOP	운영 로그레벨 INFO 유지 시 자연 감소
● 관찰	⑥⑦⑨⑩⑪	입력 규모/토글에 따라 변동 — 프로파일링으로 실측 후 판단

공통 주의: 가상 스레드(`spring.threads.virtual.enabled=true`)는 I/O 대기에는 강하지만 ①②③ 같은 **CPU-bound 작업의 처리량은 늘리지 못한다**. 이들 핫스팟의 진짜 상한은 물리 **CPU 코어 수**이므로, 스케일아웃(인스턴스 증 설) 또는 연산량 자체 절감이 답이다.

다음 단계: JFR 또는 `async-profiler` 로 위 후보들의 실제 CPU 점유율(self-time)을 측정해 본 정적 분석의 우선순위를 검증할 것. 부하는 로그인/PDF/대사 시나리오 위주로 구성.